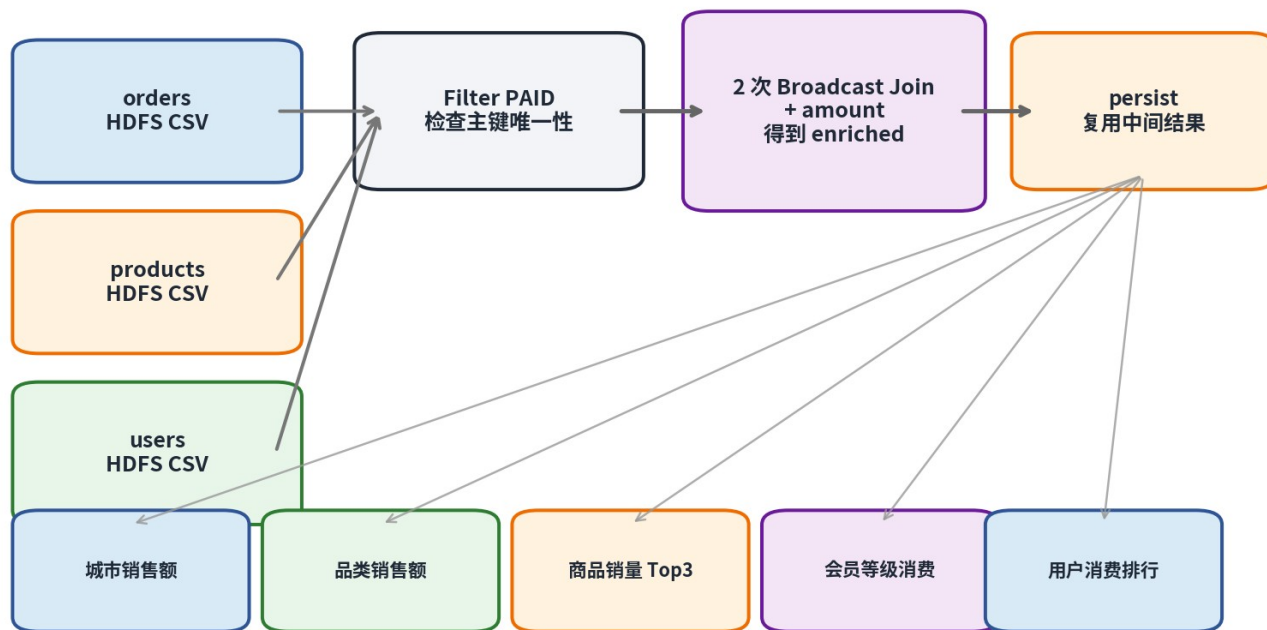


大数据综合项目实战

第 5 讲：Spark 三表离线分析 - 建立真正的离线分析层

orders + products + users → enriched → 多指标 → Parquet

Spark 三表离线分析主流水线



本讲最终目标

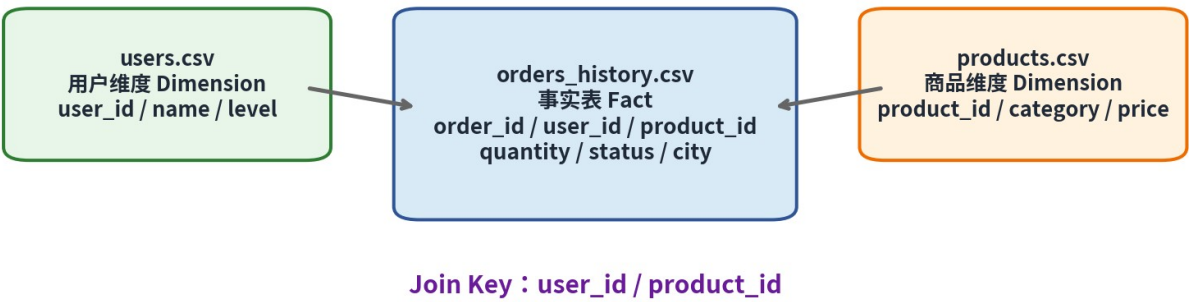
读取 HDFS 三张表，检查维度主键唯一性，过滤 PAID，完成两次 Broadcast Join，构造可复用 enriched DataFrame，生成 5 组离线分析结果，并写入 5 个 Parquet 数据集。

统一环境继续沿用前几讲：Ubuntu 22.04 + Hadoop 3.3.5 + YARN + Spark 3.x / PySpark；Windows 端通过 VS Code Remote SSH 操作 Ubuntu VM。

1. 为什么第 5 讲不再只算一个指标？

第 3、4 讲故意只做“城市 PAID 销售额”，目的是把 MapReduce 与 Spark 的执行闭环跑通。真实离线分析通常不是只算一个数字，而是从同一批事实数据和维度数据派生多个结果集，供报表、SQL、下游作业反复使用。

第5讲的数据模型：1 张事实表 + 2 张维度表



事实表记录“发生了什么”；维度表补充“这个用户/商品是谁、属于什么类别”。

事实表 vs 维度表

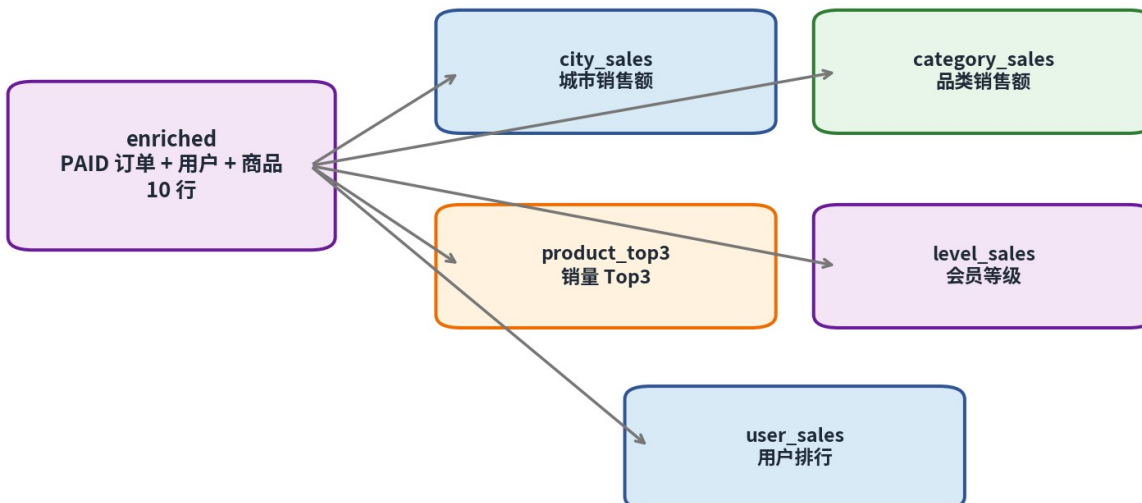
orders_history.csv 是事实表：每一行表示一次业务事实。users.csv 和 products.csv 是维度表：提供用户等级、商品类别和价格等描述信息。

表	角色	主键/关联键	本讲用途
orders_history.csv	事实表	order_id; 含 user_id/product_id	订单事实、数量、状态、城市
products.csv	商品维度	product_id	补 product_name/category/unit_price
users.csv	用户维度	user_id	补 name/level/user_city

2. 本讲最终要产出哪 5 组结果？

输出	业务问题	核心字段
city_sales	每个城市 PAID 订单数与销售额	city, paid_order_count, total_sales
category_sales	每个品类的订单数、销量与销售额	category, quantity, sales
product_top3	销量最高的 3 个商品	product_id, product_name, quantity, sales
level_sales	VIP 与 Normal 谁贡献更多销售额	level, order_count, sales
user_sales	每个用户的 PAID 订单数与消费额排行	user_id, name, level, sales

第5讲离线分析层：一个输入模型，输出多个 Parquet 数据集



每一个输出目录都是独立数据集，可被下一讲 Spark SQL、报表或其他作业继续读取。

3. 每次课固定起手式：虚拟机 → VS Code → 服务健康检查

默认虚拟机刚启动，服务未运行。先确认 VS Code 左下角是 Remote SSH 到 Ubuntu，再打开远程终端。终端提示符必须来自 Ubuntu，而不是 Windows PowerShell。

```
whoami
hostname
pwd

java -version
hadoop version | head
spark-submit --version

# 先看进程
jps

# 启动 Hadoop 服务
start-dfs.sh
start-yarn.sh

# 再检查
jps
hdfs dfsadmin -safemode get
yarn node -list
```

不要跳过这一步

如果 HDFS/YARN 根本没有起来，后面任何 Spark 报错都没有分析价值。先证明环境层健康，再进入数据层。

4. 检查三张 HDFS 输入表确实存在

```
hdfs dfs -ls /bigdata-capstone/raw/orders
hdfs dfs -ls /bigdata-capstone/raw/products
hdfs dfs -ls /bigdata-capstone/raw/users

hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv
hdfs dfs -cat /bigdata-capstone/raw/users/users.csv
```

人工快速核对：orders 应有 12 条业务记录；products 5 条；users 5 条。PAID 订单应该是 10 条。

5. 在 VS Code 中创建第 5 讲工作目录

```
cd ~/bigdata-capstone
mkdir -p spark/lecture5
cd spark/lecture5
pwd
ls -lah
```

把代码包中的 05_multi_table_offline_analysis.py 放到该目录。第一次请在 VS Code 中打开它，按本讲结构逐段阅读，不要直接双击脚本“一键跑完”。

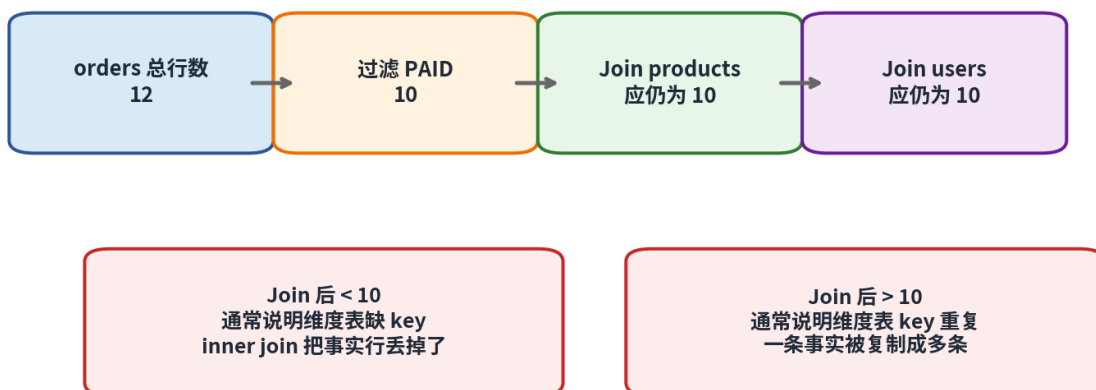
6. 第一个新能力：Join 前先检查维度主键是否唯一

这是多表分析里非常重要、但初学者最容易忽略的检查。产品表理论上一个 product_id 只能出现一次；用户表一个 user_id 也只能出现一次。如果维度主键重复，Join 会把事实表的一行复制成多行，销售额就会被重复计算。

```
products_count = products.count()
product_key_count = products.select("product_id").distinct().count()
assert product_key_count == products_count

users_count = users.count()
user_key_count = users.select("user_id").distinct().count()
assert user_key_count == users_count
```

多表 Join 最重要的证据：行数不应该莫名变化



因此先检查 users.user_id 与 products.product_id 是否唯一，再做 Join。

把 Join 行数当成业务证据

本讲固定：PAID=10；Join products 后仍应为 10；再 Join users 后仍应为 10。少了说明 key 缺失，多了说明维度 key 重复。

7. 先过滤 PAID，再做两次 Broadcast Join

过滤越早，后面 Join 处理的数据越少。products 和 users 都只有 5 行，非常适合作为广播维度表。

```
paid = orders.filter(col("status") == "PAID")

products_dim = products.select(
    "product_id", "product_name", "category", "unit_price"
)

users_dim = (
    users.select("user_id", "name", "city", "level")
    .withColumnRenamed("city", "user_city")
)

after_product = paid.join(
    broadcast(products_dim), "product_id", "inner"
)

enriched = (
    after_product.join(broadcast(users_dim), "user_id", "inner")
    .withColumn("amount", col("quantity") * col("unit_price"))
)
```

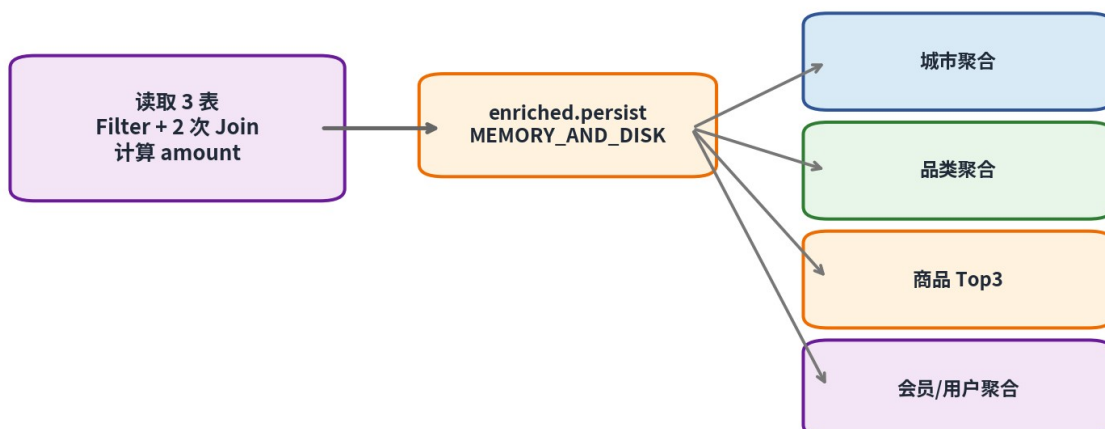
为什么 users.city 改名为 user_city?

订单表已经有 city。为了避免 Join 后出现两个含义不同但同名的 city，本讲把用户注册城市明确改成 user_city；主分析仍以订单发生城市 orders.city 为准。

8. 第二个新能力：为什么这次适合 persist？

第 4 讲中间 DataFrame 基本只走一条下游路径，强行 cache 未必有收益。第 5 讲的 enriched 会被 5 组分析重复使用，如果不缓存，Spark 可能在多个 Action 中重复读取 CSV、过滤和 Join。

为什么第5讲适合 persist，而第4讲未必需要？



同一个 enriched 被多次复用，缓存才有明确收益；只用一次的数据不要机械 cache。

```
from pyspark import StorageLevel

enriched.persist(StorageLevel.MEMORY_AND_DISK)
enriched_count = enriched.count() # 触发并物化缓存
print(enriched_count)             # expected 10
```

cache/persist 不是越多越好

只有当“同一中间结果会被多次复用”时才考虑缓存；用完后调用 unpersist()。缓存会占 Executor 存储内存，不能把所有 DataFrame 都缓存。

9. 指标一：城市 PAID 订单数与销售额

```
city_sales = (
    enriched.groupBy("city")
    .agg(
        count("order_id").alias("paid_order_count"),
        spark_sum("amount").alias("total_sales"),
    )
    .orderBy("city")
)
city_sales.show(truncate=False)
```

city	paid_order_count	total_sales
Beijing	4	6874
Guangzhou	2	676
Shanghai	3	6984
Shenzhen	1	299
TOTAL	10	14833

10. 指标二：品类销售额 - 为什么结果很有业务意义？

```
category_sales = (
    enriched.groupBy("category")
    .agg(
        count("order_id").alias("paid_order_count"),
        spark_sum("quantity").alias("total_quantity"),
        spark_sum("amount").alias("total_sales"),
    )
    .orderBy(desc("total_sales"), "category")
)
```

category	订单数	销量	销售额
Digital	4	4	12996
Accessory	4	8	1542
Education	2	5	295

这里能看到一个很直观的业务现象：Accessory 销量最高，但 Digital 销售额最高。说明“销量”与“销售额”是两个不同指标，不能混为一谈。

11. 指标三：商品销量 Top3

```
product_stats = (
    enriched.groupBy("product_id", "product_name")
    .agg(
        spark_sum("quantity").alias("total_quantity"),
        spark_sum("amount").alias("total_sales"),
    )
)

product_top3 = (
    product_stats
    .orderBy(desc("total_quantity"), desc("total_sales"), "product_id")
    .limit(3)
)
```

排名	商品	total_quantity	total_sales
1	P002 Mouse	5	645
2	P003 Book	5	295
3	P004 Keyboard	3	897

为什么要写完整排序规则？

P002 与 P003 的销量都为 5。如果只按 quantity 排序，二者顺序不一定稳定；本讲再按 sales、product_id 做 tie-break，保证验收结果可重复。

12. 指标四：VIP / Normal 会员等级消费

```
level_sales = (
    enriched.groupBy("level")
    .agg(
        count("order_id").alias("paid_order_count"),
        spark_sum("amount").alias("total_sales"),
    )
    .orderBy("level")
)
```

level	paid_order_count	total_sales
Normal	5	7601
VIP	5	7232

不要预设“VIP 一定消费更高”。这一批教学数据里 Normal 的总销售额反而更高。数据分析的任务是让结果回答问题，而不是让结果迎合我们的直觉。

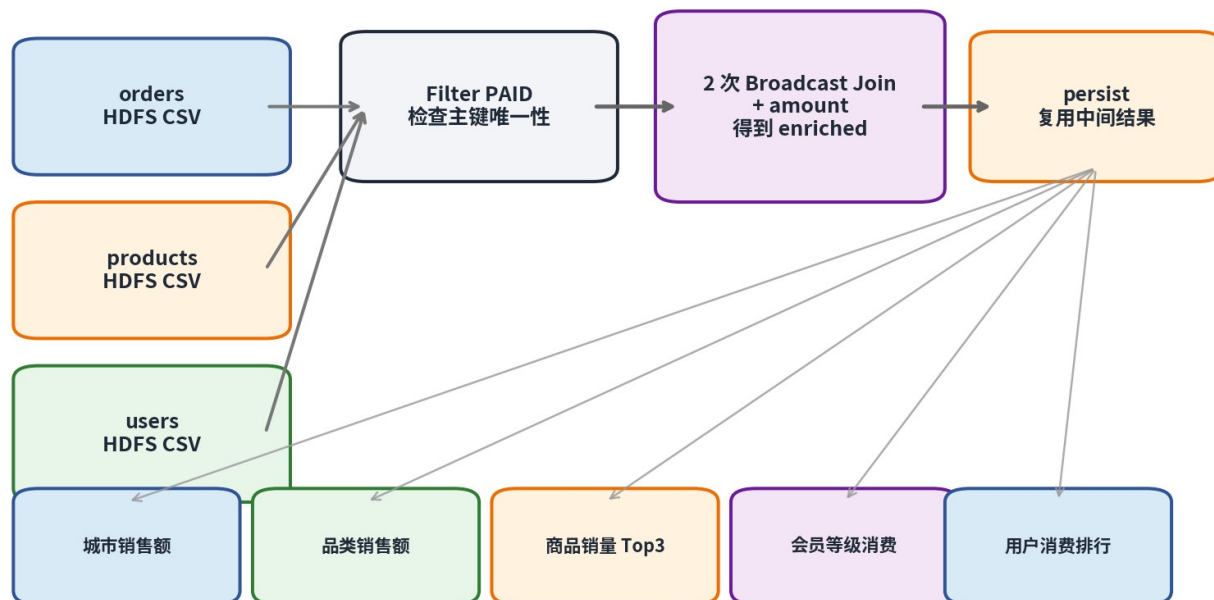
13. 指标五：用户消费排行

```
user_sales = (
    enriched.groupBy("user_id", "name", "level")
        .agg(
            count("order_id").alias("paid_order_count"),
            spark_sum("amount").alias("total_sales"),
        )
        .orderBy(desc("total_sales"), "user_id")
)
```

user	level	订单数	消费额
U002 LiSi	Normal	3	6984
U001 ZhangSan	VIP	2	6257
U003 WangWu	VIP	2	676
U004 ZhaoLiu	Normal	2	617
U005 SunQi	VIP	1	299

14. 把 5 组指标重新放回同一条执行逻辑

Spark 三表离线分析主流水线



三张表只需要构造一次 enriched。随后不同指标各自产生 groupBy / orderBy / limit 等执行分支。真实 Spark Application 中可能形成多个 Job / Stage；不要把“一个 Python 脚本”误解为“只会产生一个 Spark Job”。

15. 看 explain：这次为什么会出现两个 Broadcast Join？

```
level_sales.explain(mode="formatted")
```

不同 Spark 版本的 Physical Plan 文本会略有差异。本讲不要死背整段输出，而是确认是否能找到：读取三张 CSV 的 Scan、两个 BroadcastHashJoin、聚合相关 HashAggregate，以及 groupBy 需要的 Exchange。

看到 Exchange 就问 “谁需要重新见面？”

例如 level_sales 按 level 聚合，不同 Partition 中属于同一个 level 的记录需要汇聚，因此会有跨 Partition 的重分布。

16. 保姆式提交：Spark on YARN

```
cd ~/bigdata-capstone/spark/lecture5
ls -lh 05_multi_table_offline_analysis.py

# 第一次先人工确认三张输入都存在
hdfs dfs -test -e /bigdata-capstone/raw/orders/orders_history.csv && echo OK-orders
hdfs dfs -test -e /bigdata-capstone/raw/products/products.csv && echo OK-products
hdfs dfs -test -e /bigdata-capstone/raw/users/users.csv && echo OK-users

# 清理旧输出（程序内部用 overwrite，但课堂上仍建议明确知道输出位置）
hdfs dfs -rm -r -f /bigdata-capstone/output/spark/lecture5

spark-submit --master yarn --deploy-mode client --num-executors 1 --executor-cores 1 --executor-memory 1g
--driver-memory 1g 05_multi_table_offline_analysis.py
```

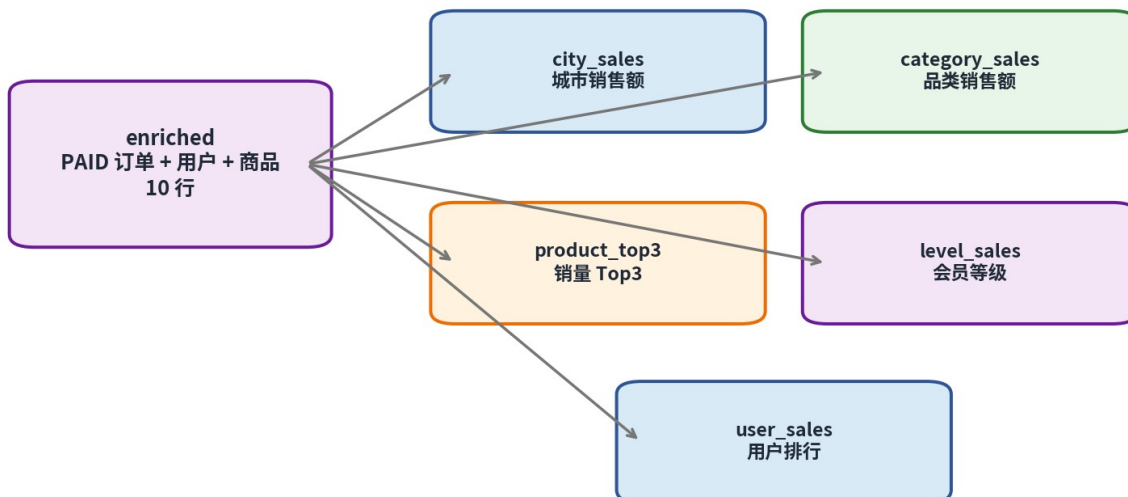
运行时要观察什么？

不是只等最后的 [PASS]。依次观察：输入行数 → 主键唯一性 → PAID=10 → 两次 Join 行数都为 10 → 5 组结果 → explain → 5 个 Parquet 写出 → 回读 TOTAL=14833。

17. HDFS 输出目录：第一次形成真正的“离线分析层”

```
hdfs dfs -ls /bigdata-capstone/output/spark/lecture5
hdfs dfs -ls -R /bigdata-capstone/output/spark/lecture5 | head -n 100
```

第5讲离线分析层：一个输入模型，输出多个 Parquet 数据集

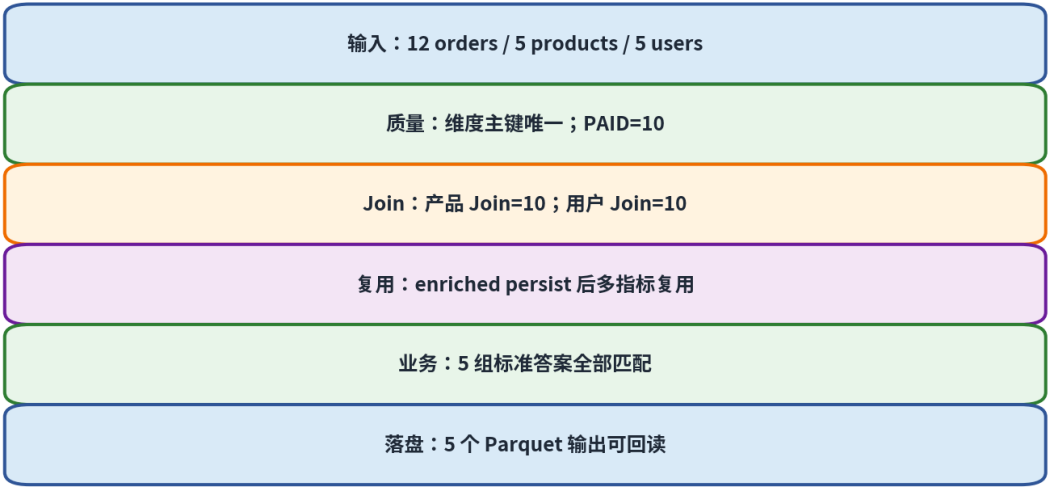


每一个输出目录都是独立数据集，可被下一讲 Spark SQL、报表或其他作业继续读取。

注意：这些都是 Parquet 数据集目录，不要用 `hdfs dfs -cat` 去硬读 `part-*.parquet`。程序已经用 `spark.read.parquet()` 回读 `city_sales` 做最终验证。后续也可以用 Spark SQL / DataFrame 继续查询这些结果。

18. 第 5 讲的六层证据链

第5讲验收证据链



为什么标准答案越来越多？

综合项目进入多指标阶段后，“TOTAL=14833”已经不够。我们现在同时验收城市、品类、Top3、会员等级、用户排行，能更早发现 Join、分组或排序逻辑错误。

19. 常见错误：多表分析最容易错在哪里？

现象	第一检查项	典型原因
Join 后 < 10	维度表缺 key	inner join 丢掉事实行
Join 后 > 10	维度主键是否唯一	重复 key 导致事实行倍增
AnalysisException ambiguous city	列重名	users.city 与 orders.city 未重命名
结果越来越慢	是否重复计算 enriched	多 Action 但未合理 persist
Executor 内存紧张	persist / 数据规模	缓存过多或缓存级别不合适
Top3 顺序偶尔变	排序规则	并列项没有稳定 tie-break
Parquet cat 不可读	文件格式	Parquet 不是文本

20. 故意制造 4 个错误，训练排错

实验 A：复制一行 products 的 P002

再运行。理论上 product_id distinct count < products count，程序会在 Join 前直接 assert 失败。理解为什么“主键质量检查”比事后发现销售额翻倍更好。

实验 B：删除 users 中 U003

Join users 后 enriched_count 会小于 10。定位到用户维度缺失，而不是怪 Spark/YARN。

实验 C：去掉 user_city 重命名

观察 Join 后同名 city 字段带来的歧义，理解列语义管理的重要性。

实验 D：注释 persist / unpersist

业务结果不变。观察 Spark UI / 日志中重复计算行为，理解“性能改动”和“正确性改动”是两件事。

21. 练习题

题号	问题
1	为什么 orders 是事实表，而 users/products 是维度表？
2	为什么维度表主键重复可能让销售额被重复计算？
3	为什么第 5 讲比第 4 讲更适合 persist？
4	Accessory 的销量比 Digital 高，为什么销售额却远低于 Digital？
5	如果 products 增长到 200 GB，还应该强制 broadcast 吗？
6	如果想分析“用户注册城市”而不是“订单发生城市”，应该用哪个字段？

22. 参考答案

题号	答案
1	事实表记录订单事件；维度表提供用户/商品描述属性，通常通过外键与事实关联。
2	一个事实命中维度表两行时，Join 结果会复制成两行，后续 sum 会重复累计。
3	enriched 被 5 个下游指标重复使用，缓存可避免重复读表、过滤和 Join。
4	Digital 商品单价远高于 Accessory；销量与销售额衡量的不是同一件事。
5	不应该。广播超大表会造成网络与 Executor 内存压力，应让优化器选择其他 Join 策略或重新设计。
6	user_city。orders.city 表示订单发生城市；本讲已主动重命名避免语义混淆。

23. 本讲最低闭环能力

- 能解释事实表、维度表和 Join Key。
- 能在 Join 前检查维度主键唯一性。
- 能解释 Join 后行数少了/多了分别意味着什么。
- 能用两次 Broadcast Join 构造 enriched DataFrame。
- 能说明为什么同名 city 要重命名为 user_city。
- 能判断什么情况下 persist 有明确收益。
- 能从同一 enriched 产出城市、品类、Top3、等级、用户 5 组指标。
- 能自动对照标准答案，而不是只看 Application Success。
- 能解释 groupBy / orderBy 可能引入的 Shuffle。
- 能把 5 组结果写为 Parquet，并在 HDFS 中确认数据集存在。

24. 课后停止服务

```
# 确认没有仍在运行的 YARN Application
yarn application -list

stop-yarn.sh
stop-dfs.sh
jps
```

25. 下一讲预告：从离线文件转向实时事件

第 6 讲开始进入实时链：我们会创建 Kafka Topic，把 order_events.jsonl 里的 CREATE / PAY / CANCEL 事件逐条发送到 Kafka，并用命令行 Producer/Consumer 观察 Partition、Offset、Consumer Group 和 Lag。那一讲先只把 Kafka 流跑通，不急着接 Flink。

为什么先单独把 Kafka 跑通？

实时链也坚持分层闭环：先证明“消息系统能稳定生产、消费、记录 Offset”，再接 Flink。否则 Kafka 和 Flink 同时出问题时，很难定位是哪一层。